our overall output and demonstrating the scalability of our approach to development. We will focus on training our new hires on iteration and process improvements, saving team members time. We will also review the best metrics to focus on and are considering moving back to an overall MR Rate measure (from an authorship one). Doing so will help us measure the efficiency of our responsiveness to our peers for the company and the community.

Usability

User experience is a continued focus area for FY24. Millions of customers use GitLab so UX improvements can have a huge collective impact across all of these individuals.

Development team members should also constantly suggest and investigate how to improve the overall user experience of the product. These can range from enhancing performance (actual and perceived), suggesting new technologies, solving user experience issues efficiently, etc.

Product with a customer focus

Lastly, we will continue our strong partnership with Product to make GitLab the best, most complete DevSecOps platform on the planet. While we continue adding features to the product we must also work to identify technical debt and bring it to the prioritization discussion. We expect that Engineering managers are already addressing technical debt that is group specific with their Product Manager.

This coordination and prioritization requires a lot of work and effort to provide the right data and make the right decisions. We will focus on a variety of factors, but top of mind will be our parent department's direction to be customer focused.

Diversity

We will follow our parent department Engineering lead.

Organizational responsibilities

The Expansion department includes the following sub-departments and groups:

- Secure
- Software Supply Chain Security
- Fulfillment
- Growth

Team Members

The following people are permanent members of the Development Department:

{{< team-by-departments departments="Expansion,Fulfillment,Growth,Sec" manager="VP of Incubation Engineering" >}}

Stable Counterparts

The following members of other functional teams are our stable counterparts:

<!-- <%= stablecounterparts(roleregexp: /[,&] Development/, directmanagerrole: 'VP of Incubation Engineering') %> -->

Development-Specific People Processes

Promotion Process

Aligned with the company-wide promotion cadence, Development utilizes a quarterly process to collect, validate, approve, review all promotion proposals prior to them being added via the company-wide process. The goal of this quarterly promotion projection and review is to:

- Promote the right people at the right time
- Maintain a high bar for promotions
- Ensure predictability and intentionality with promotions
- Ensure alignment with overall company promotion rate
- Add another layer of review to reduce bias in the promotion process

Development adheres to the company-wide quarterly timeline outlined here as our SSOT.

The Development Department has an additional formal step built in to our promotion process beyond what the company is currently adhering to through our peer review process. Ahead of the commencement of the Calibration stage of our process, all promotion documents should be peer reviewed by a Senior Manager or Director. The due date to complete the peer review is before the scheduled Calibration session.

FY'23 Calibration sessions:

1. FY24-Q1: January 13, 2022
2. FY24-Q2: April 7, 2022
3. FY24-Q3: June 30, 2022
4. FY24-Q4: October 5, 2022

Calibration session attendees are the following team members: Senior Managers, Directors, Sr. Directors, VP, and Development's aligned People Business Partner. Leaders are welcome to conduct Calibration sessions prior to the scheduled sessions above with their sub-departments as well (though this is not a requirement).

In addition to the company-wide calibration preparation, for the Development department we also ask that leaders come prepared to discuss:

1. Status of maintainership
2. Most recent talent assessment

Calibration Agenda

In order for calibration to be effective it's important that all participants have had an opportunity to review promotional documents and summaries ahead of the meeting.

The calibration agenda will consist of the following for each candidate:

1. General Information

- Promotion Doc Peer Reviewer(s)
- Link to GitLab Profile

1. Core accomplishments (list 2)
1. Improvement areas (list 2)

- The promotion document outlines strengths, but we also want to highlight how we will support a team member's development areas at the next level.

1. Cross-functional Feedback

- As our business goals and initiatives become increasingly cross-functional, managers should have a picture of how their team member collaborates effectively within their immediate teams, and with their core cross-functional partners and stakeholders.

1. Questions/feedback?

Please aim to be concise and crisp in the calibration agenda summary for each candidate. Leaders are able to reference promotion documents for details, while the calibration agenda summary is meant to be a snapshot of key points to help facilitate discussion and provide an overview for the group.

To allow time for review and the addition of questions/feedback, summaries should be included in the agenda no less than one week prior to the Development Leadership scheduled calibration date.

In line with our guidance on feedback, feedback should be regular and ongoing. Calibration sessions are meant to discuss team member promotion readiness and calibrate promotions across the department, but they should not replace the regular and ongoing feedback provided throughout the year. Any relevant feedback should be given promptly and not wait until talent assessments or promotion calibration. This will ensure that both the team member and their manager have an opportunity to address feedback in a timely manner.

Talent Assessment Process

Talent Assessment Process guidelines specific for the Expansion Development Department is documented in this handbook page.

How we hire contractors

In this handbook page we document the process that the development departments follow, including planning budget, candidate sourcing, interview process, contracting and onboarding.

How We Work

Onboarding

Welcome to GitLab! We are excited for you to join us.

Here are some curated resources to get you started:

- Joining as an Engineer
- Joining as an Engineering Manager

Cross-Functional Metrics

Link to dashboard

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/IssueTypesDetail/OpenIssuesDashboard" >}}
{{< /tableau >}}

{{% include "includes/cross-functional-prioritization.md" %}}

Cross-Functional Collaboration

Working across Stages

Issues that impact code in another team's product stage should be approached collaboratively with the relevant Product, UX, and Engineering managers prior to work commencing, and reviewed by the engineers responsible for that stage.

We do this to ensure that the team responsible for that area of the code base is aware of the impact of any changes being made and can influence architecture, maintainability, and approach in a way that meets their stage's roadmap.

Architectural Collaboration

At times when cross-functional, or cross-departmental architectural collaboration is needed, the GitLab Architecture Evolution Workflow should be followed.

Follow the Sun Coverage

When cross-functional collaboration is required across global regions and time zones, it is recommended to adopt the Follow the Sun Coverage approach to ensure seamless global collaboration.

Decisions requiring approvals

At GitLab we value freedom and responsibility over rigidity. However, there are some technical decisions that will require approval before moving forward. Those scenarios are outlined in our required approvals section.

Security Vulnerability Handling

1. The development groups who introduce or consume the dependency of concern (e.g. gems, libs, base images, etc.) are responsible for resolving vulnerabilities detected against the dependency.
2. For business selected vendors that provide base images (RHEL's UBI8 for example), we need to

wait for their patches, or need to log Deviation Request (DR) as viable resolutions. The VulnMapper, an automation developed by the Threat Management team, can create vendor dependency DRs to a large extent, but there are still cases that DR needs to be reported manually.
3. The assigned development group can redirect issues if the initial assignment was inaccurate, following the processes for shared responsibility issues and/or Shared responsibility functionality.

Development Headcount planning

Development's headcount planning follows the Engineering headcount planning and long term profitability targets. Development headcount is a percentage of overall engineering headcount. For FY20, the headcount size is 271 or ~58% of overall engineering headcount.

We follow normal span of control both for our managers and directors of 4 to 10. Our sub-departments and teams match as closely as we can to the Product Hierarchy to best map 1:1 to Product Managers.

Development Staff Meeting

While we try to work as much as possible async, the Development department leadership does meet synchronously on a cadence of weekly. This meeting coordinates initiatives, communicates relevant information, discusses more difficult decisions, and provides feedback on how we are progressing as an organization. As part of this meeting, we discuss our culture of reliability monthly. This was part of the agenda spawned from an initiative we took up in August of 2021. We want to make sure we keep the organization healthy when thinking about reliability in every part of our work.

Daily Duties for Engineering Directors

This section applies to those who report to the VP of Development

The following is a non exhaustive list of daily duties for engineering directors, while some items are only applicable at certain time, though.

1. Review engineering metrics
    1. Development Department Performance Indicators
    1. Sub-department Performance Indicators
    1. Dev
    1. Enablement
    1. Fulfillment
    1. Growth
    1. Ops
    1. Secure
    1. Software Supply Chain Security
1. Review hiring dashboards
1. Personal todo list
1. Personal GitLab board(s) if any
1. Working groups that the director drives or participates in
    1. Action items in agenda documents
    1. Issue boards

    1. Slack channel
1. Infradev triage
    1. Follow up open questions and ensure appropriate handling of issues with regard to priority and severity
    1. Agenda document
    1. Infradev board
1. Performance refinement
    1. Follow up open questions and ensure appropriate handling of issues with regard to priority and severity
    1. Agenda document
    1. Performance board
1. Infrastructure Development Escalations
    1. Triage new issues, enhance Issue details and ensure appropriate handling based on priority and severity
    1. Sync discussions for infradev Issues are part of the GitLab SaaS Weekly Meeting
    1. Agenda document
    1. Infradev board
1. Follow active Engineering Global Prioritization(s) that the director sponsors
    1. Standup/status update document
    1. Issue board
1. Holiday Emergency Contact Rotations
1. Review and approve security approvals for the GitLab project when required and informing the security engineering team when a security risk is accepted rather than being resolved prior to approval.

Developing and Tracking OKRs

In general, OKRs flow top-down and align to the company and upper level organization goals.

Managers and Directors

For managers and directors, please refer to a good walk-through example of OKR format for developing team OKRs. Consider stubbing out OKRs early in the last month of the current quarter, and get the OKRs in shape (e.g. fleshing out details and making them SMART) no later than the end of the current quarter.

It is recommended to assess progress weekly.

1. Append the percentage score to the subject of Objective epics and Key Result issues.
1. Set the Health status of epics and issues.
1. In the case where weekly assessment is impractical, an assessment shall be made by the end of each month.

Staff Engineers, Distinguished Engineers, and Fellows

Below are tips for developing individual's OKRs:

1. Align OKRs to team goals. However, it's unnecessary to derive from all organizational OKRs. Simply decide what makes sense to your personal situation.
1. Follow the same timeline of managers and directors, i.e. stubbing out early and bring OKRs

in shape by the end of the current quarter.
1. Refer to the same walk-through example of OKR format.
1. Make SMART OKRs - Specific, Measurable, Achievable, Relevant, Time-bound.
1. Follow the same progress assessment instructions above.

Examples

1. Engineering
1. Development

Engineering Allocations and Tracking

Engineering Allocation require us to track goals with more diligence and thought. We need confidence that we're making correct decisions and executing well to these initiatives. As such, you will see us reviewing these more closely than other initiatives. We will meet on a cadence to review these initiatives and request additional reporting to support the process. Possible requests for additional data:

1. Demos
1. GitLab Roadmaps
1. GitLab Architecture Workflow
1. Dogfooding of features we think may be useful

We will hold Engineering Allocation Checkpoints on a cadence. The recommended cadence is weekly.

Roadmaps for Engineering Allocations

We track Engineering Allocation roadmaps. To use this effectively, roadmaps must have correct dates for their epic and weights assigned to issues. If a team does not normally use weights, then assign each issue a weight of 1 (all issues are equal).

Team allocation measurement

Each team needs to demonstrate how their allocation is being used. This is done to verify we are not over/under investing for a given initiative. This can be done via assignment (people assigned to work) and/or issues assigned. We will track issues and MRs and see as a percentage how that compares to the overall teams work.

Ownership of Shared Services and Components

The GitLab application is built on top of many shared services and components, such as PostgreSQL database, Redis, Sidekiq, Prometheus and so on. These services are tightly woven into each feature's rails code base. Very often, there is need to identify the DRI when demand arises, be it feature request, incident escalation, technical debt, or bug fixes. Below is a guide to help people quickly locate the best parties who may assist on the subject matter.

Ownership Models

There are a few available models to choose from so that the flexibility is maximized to streamline what works best for a specific shared service and component.

1. Centralized with Specific Team
    1. A single group owns the backlog of a specific shared service including new feature requests, bug fixes, and technical debt. There may or may not be a counterpart Product Manager.
    1. The single group is a specific team, meaning there is an engineering manager and all domain owner individuals reside in this team. The DRI is the engineering manager.
    1. This single group is expected to collaborate closely and regularly in grooming and planning backlog.
    1. This model may require consensus from the Product Management counterpart.
    1. This model may fit a subject domain that experiences active development.
1. Centralized with Virtual Team
    1. A single group owns the backlog of a specific shared service including new feature requests, bug fixes, and technical debt. There may or may not be a counterpart Product Manager.
    1. The single group is a virtual team, meaning it consists of engineers from various engineering teams, for example maintainers or subject matter experts. Typically there isn't an engineering manager for this virtual team. The DRI is an appointed person in the group who may not necessarily be an engineering manager.
    1. This single group is expected to collaborate closely and regularly in grooming and planning backlog.
    1. This model may fit a subject domain that's in maintenance mode.
1. Collectives
    1. Collectives consist of individuals from existing teams who voluntarily rally around a shared interest or responsibility, but unlike Working Groups may exist in perpetuity. The shared interest could be a specific technology or system. Collective members feel a collective responsibility to weakly own, improve upon or otherwise steer the subject they govern.
    1. This is a weaker form of the Virtual Team but introduces more structure than a fully decentralized model. It can be appropriate when some form of ownership is desirable where the subject has cross-cutting impact and wide reach and cannot clearly be allocated to any specific team.
    1. Collectives do not have product or engineering managers, they are fully self-governed.
    1. Members of the Collective sync regularly and keep each other informed about the shared interest. Problem areas are identified and formalized in the Collective, but are not logged into a Collective backlog. Instead a DRI is assigned who should put the task forward to the team with the greatest need for the problem to be resolved. This is to ensure that work is distributed fairly and that there are no two backlogs that compete with each other for priorities.
    1. Collectives work best when they consist of a diverse set of individuals from different areas of product and engineering. They double as knowledge sharing hubs where information is exchanged from across teams in the Collective first, and then carried back by the individuals to their specific teams.
1. Decentralized
    1. The team who implements specific functions or utilizes certain features of the shared services is responsible for their changes from local development environment to production deployment to continued maintenance post-deployment. There is not a development-wide single DRI who owns a portion or the entirety of a shared service.
    1. A specialty team may exist for specific subject domains, however their role is to enable scalability, availability, and performance by building a solid foundation and great tools for testing and troubleshooting for other engineering teams, while they are not responsible for

gating every single change in the subject domain.

Shared Services and Components

The shared services and components below are extracted from the GitLab product documentation.

Service or Component	Sub-Component	Ownership Model	DRI (Centralized Only)	Ownership Group (Centralized Only)	Additional Notes
Alertmanager		Centralized with Specific Team	@twk3	Distribution	Distribution team is responsible for packaging and upgrading versions. Functional issues can be directed to the vendor.
Certmanager		Centralized with Specific Team	@twk3	Distribution	Distribution team is responsible for packaging and upgrading versions. Functional issues can be directed to the vendor.
Consul					
Container Registry		Centralized with Specific Team	@dcroft	Package	
Email - Inbound					
Email - Outbound					
GitLab K8S Agent		Centralized with Specific Team	@nicholasklick	Configure	
GitLab Pages		Centralized with Specific Team	@vshushlin	Knowledge	
GitLab Rails		Decentralized			DRI for each controller is determined by the feature category specified in the class. app/controllers and ee/app/controllers
GitLab Shell		Centralized with Specific Team	@andr3	Create:Source Code	Reference
HAproxy		Centralized with Specific Team		Infrastructure	
Jaeger		Centralized with Specific Team	@dawsmith	Infrastructure:Observability	Observability team made the initial implementation/deployment.
LFS		Centralized with Specific Team	@andr3	Create:Source Code	
Logrotate		Centralized with Specific Team	@twk3	Distribution	Distribution team is responsible for packaging and upgrading versions. Functional issues can be directed to the vendor.
Mattermost		Centralized with Specific Team	@twk3	Distribution	Distribution team is responsible for packaging and upgrading versions. Functional issues can be directed to the vendor.
MinIO		Decentralized			Some issues can be broken down into group-specific issues. Some issues may need more work identifying user or developer impact in order to find a DRI.
NGINX		Centralized with Specific Team	@twk3	Distribution	
Object Storage		Centralized with Specific Team	@lmcandrew	Scalability::Frameworks	
Patroni	General except Geo secondary clusters	Centralized with Specific Team	@twk3	Distribution	
	Geo secondary standby clusters	Centralized with Specific Team	@luciezhao	Geo	
PgBouncer		Centralized with Specific Team	@twk3	Distribution	
PostgreSQL	PostgreSQL Framework and Tooling	Centralized with Specific Team	@alexives	Database	Specific to the development portion of PostgreSQL, such as the fundamental architecture, testing utilities, and other productivity tooling
	GitLab Product Features	Decentralized			Examples like feature specific schema changes and/or performance tuning, etc.
Prometheus		Decentralized			Each group maintains their own metrics.

Puma		Centralized with Specific Team	@pjphillips	Cloud Connector	
Redis		Decentralized			DRI is similar to Sidekiq which is determined by the feature category specified in the class. app/workers and ee/app/workers
Sentry		Decentralized			DRI is similar to GitLab Rails which is determined by the feature category specified in the class. app/controllers and ee/app/controllers
Sidekiq		Decentralized			DRI for each worker is determined by the feature category specified in the class. app/workers and ee/app/workers
Workhorse		Centralized with Specific Team	@andr3	Create:Source Code	

Learning Resources

For a list of resources and information on our GitLab Learn channel for Development, consult this page.

Continuous Delivery, Infrastructure and Quality Collaboration

In late June 2019, we moved from a monthly release cadence to a more continuous delivery model. This has led to us changing from issues being concentrated during the deployment to a more constant flow. With the adoption of continuous delivery, there is an organizational mismatch in cadence between changes that are regularly introduced in the environment and the monthly development cadence.

To reduce this, infrastructure and quality will engage development via SaaS Infrastructure Weekly and Performance refinement which represent critical issues to be addressed in development from infrastructure and quality.

Refinement will happen on a weekly basis and involve a member of infrastructure, quality, product management, and development.

Global Prioritization

Execution of a Global prioritization can take many forms. This is worked with both Product and Engineering Leadership engaged. Either party can activate a proposal in this area. The options available and when to use them are the following:

- Rapid action - use when reassignment isn't necessary, the epic can have several issues assigned to multiple teams
- Borrow - use when a temporary assignment to a team is required to help resolve an issue/epic
- Realignment - use when a permanent assignment to a team is required to resolve ongoing challenges

Email alias and roll-up

1. Available email alias (a.k.a. Google group):
Managers, Directors, VP's teams: each alias includes everyone in the respective organization.
1. Naming convention:
team@gitlab.com, examples below -
    - Managers: configure-be@gitlab.com includes all the engineers reporting to the Configure

backend engineering manager.
    - Directors: ops-section@gitlab.com includes all the engineers and managers reporting to the director of engineering, Ops.
    - VP of Development: development@gitlab.com includes all engineers, managers, and directors reporting to the VP of Development.
1. Roll up:
Teams roll up by the org chart hierarchy -
    - Engineering managers' aliases are included in respective Sub-department aliases
    - Sub-department aliases are included in Development alias

Working with Support

When Development collaborates with Support it provides invaluable insight into how customers are using the product and the challenges they run into. A few tips to make the process efficient:

- Get access to Zendesk so you view the question and communication from customers.
- Always write answers in a way that they can be "cut-and-pasted" and sent to a customer.
- Reference documentation in your responses and make updates to GitLab documentation when needed.
- Refer to existing issues and epics to reiterate our transparency value and to invite participation from the customer.
- If you are unclear about the support-development collaboration process or workflow then please refer to the handbook page how to use gitlab.com to request help from the GitLab development team

Incident Management

Team members in some job families contribute to incident management directly through an on-call schedule for Incident Managers.
Team members should complete onboarding so they can be added to the schedule when needed. These frequently asked questions cover exemptions and changing shifts.

- Incident Management process
- Incident Manager On Call onboarding

Development Escalation Process

- General information
- Process outline

Reducing the impact of far-reaching work

Because our teams are working in separate groups within a single application, there is a high potential for our changes to impact other groups or the application as a whole. We have to be cautious not to inadvertently impact overall system quality but also availability, reliability, performance, and security.

An example would be a change to user authentication or login, which might impact seemingly unrelated services, such as project management or viewing an issue.

Far-reaching work is work that has wide-ranging, diffuse implications, and includes changes to areas which will:

1. be utilized by a high percentage of users
1. impact entire services
1. touch multiple areas of the application
1. potentially have legal, security, or compliance consequences
1. potentially impact revenue

If your group, product area, feature, or merge request fits within one of the descriptions above, you must seek to understand your impact and how to reduce it. When releasing far-reaching work, use a rollout plan. You might additionally need to consider creating a one-off process for those types of changes, such as:

- Creating a rollout plan procedure
  - Consider how to reduce the risk in your rollout plan
  - Document how to monitor the rollout while in progress
  - Describe the metrics you will use to determine the success of the rollout
  - Account for different states of data during rollout, such as cached data or data that was in a previously valid state
- Requiring feature flag usage (example)
- Changing a recommended process to a required process for this change, such as a domain expert review
- Requesting manual testing of the work before approval

Identified areas

Some areas have already been identified that meet the definition above, and may consider altered approaches in their work:

Area	Reason	Special workflows (if any)
Database migrations, tooling, complex queries, metrics	impact to entire application The database is a critical component where any severe degradation or outage leads to an S1 incident.	Documentation
Sidekiq changes (adding or removing workers, renaming queues, changing arguments, changing profile of work required)	impact to multiple services Sidekiq shards run groups of workers based on their profile of work, eg memory-bound. If a worker fails poorly, it has the potential to halt all work on that shard.	Documentation
Redis changes	impact to multiple services Redis instances are responsible for sets of data that are not grouped by feature category. If one set of data is misconfigured, that Redis instance may fail.	
Package product areas	high percentage of traffic share	
Gitaly product areas	high percentage of traffic share	
Create: Source Code product areas	high percentage of traffic share. Special attention	

should be paid to Protected Branches, CODEOWNERS, MR Approvals, Git LFS, Workhorse and the git over SSH / gitlab-sshd interfaces. Please contact the EM (@seancarroll) or PM (@tlinz) if you are unsure. | |
Pipeline Execution product areas	high percentage of traffic share	Documentation
Authentication and Authorization product areas	touch multiple areas of the application	Documentation
Compliance product areas	potentially have legal, security, or compliance consequences	Code Review Documentation
Workspace product areas	touch multiple areas of the application	Documentation
Specific fulfillment product areas	potentially impact revenue	
Runtime language updates	impacts to multiple services	Ruby Upgrade Guidelines
Application framework updates	impacts to multiple services	Rails Upgrade Guidelines
Navigation	impact to entire application	Proposing a change that impacts navigation

AI-powered stakeholders

This section provides an overview of all teams invested in implementing and maintaining AI features. Our Duo initiative is a cross-category effort.

These are the stakeholders:

Team	Stake
Create:Remote Development	Owns the WebIDE (maintainers)
Editor Extensions	Maintains the GitLab Workflow VS Code Extension (maintainers), JetBrains, Neovim, Visual Studio extensions and the language server. Also contributes with UX improvements for Code Suggestions within GitLab Workflow.
Enablement:Cloud Connector (@mkaeppler, @nmilojevic1)	AI-Assissted for Self-Managed
AI Framework	Abstraction Layer for GitLab Chat, Code Suggestions and other AI capabilities
Duo Chat	GitLab Chat for VSCode and WebIDE
Create:Code Creation	Code Suggestions
AI Model Validation Group	Suggested Reviewer, Code Suggestions AI Gateway functionality, Evaluating and tuning ML Models
Infrastructure	Code Suggestions AI Gateway scalability

ClickHouse Datastore usage

ClickHouse usage by Monitor:Platform Insights group

Customer Account Escalation coordination

If development is the DRI or actively participating in a Customer Account Escalation, consider the following:

- Be careful to not make commitments to customers without first talking to product management and development leaders to confirm the impact that commitment may have on other commitments.
- The customer will want to know when they can see the benefits of a change. They may not be familiar with GitLab practices for tracking and predicting due dates and milestones. Also, they may not be familiar with our workflows and associated labels nor the predictability of code review timelines, different timelines on releases to GitLab.com compared with releases for self-hosted customers and our use of feature flags.

markdown
Customers often don't rely on asynchronous communication at the level that GitLab does. Educate the customer on our practices and adapt to find a combined asynchronous and synchronous communication method and cadence that works for everyone.
Encourage customers to collaborate with us in epics, issues, and merge requests of interest. Keep in mind that they may not have access to ones that are confidential and/or may not be comfortable or able to collaborate with us in this public forum.
Consider utilizing Google documents to collaborate with the customer as a backup for collaboration via epics, issues, and merge requests.
Consider utilizing a shared Slack channel to collaborate, adding the customers to our slack via "one Slack channel access requests". Example
In meetings, tell customers why we like to record them and ask if they are OK with doing so. Consider using Chorus for scheduling the recordings to address legal requirements for recording meetings with customers.
In meetings, tell customers why we take notes before, during, and after the meeting, as it may not be natural for them to collaborate in this way.
Make sure the appropriate priority label is applied to all issues being tracked by the customer.
In the agenda for recurring meetings, track the items tracked by the customer in priority order at the top and review the status, next steps, customer DRI, and GitLab DRI for each. Discuss in the meeting periodically.
Remind GitLab team members in Slack to update the status of items they are the DRI for before recurring meetings.
Post a link to the meeting notes and recording in a Slack channel for the customer escalation, so those who did not attend know that the notes and recording are available for review.
When there is an action item for someone in a meeting (whether they are present or not), tag them in an issue or MR (or in Slack) so they will see it.

FY24 Team Building "Fun" Budget

Overview of the budget

As part of the FY24 team building budget available to each division, Engineering is allowing team members and teams to self-organize and propose team building events to use the budget. Development teams may apply to use the budget for team building events.

The budget limit per team member in FY24 is $500 and a team member must be part of an approved

application to submit expenses related to a team building event.

Examples of how it may be used

The team building budget may be used for a variety of activities, including but not limited to:

- In-person meetups with teammates and other department team members. Note that the budget is limited and may not cover the full cost of travel and lodging.
- Expensing a meal for a virtual team event.
- Credits for team members to buy GitLab swag.
- Coordinating a social event with team members who are already attending a conference.

What it is not intended for
- Conference registration and travel. These should be applied for separately using the Growth and Development benefit application process.

Application Process

- Create an issue in your team's issue tracker or the Team Member Socials issue tracker containing the following details:
    - Event/Usage description
    - Event date
    - Quarter when expenses will bu submitted. This can be different from the quarter in which the event will take place.
    - Number of team members attending
    - Total amount requested in USD.
- Ask your stage or sub-department leader to add the issue details to the FY24 Fun Budget Application Google Spreadsheet and ping the VP Development for approval.
- Once approval is granted or denied, the stage/sub-department leader will update the spreadsheet with the status and inform the team members who applied.

Common Links

Development department board
Current OKR's
Slack channel expansion-development
Manager Notes

SECTION: engineering/fast-boot.md

A Fast Boot is an event that gathers the members of a team or group in one physical location to work together and bond in order to accelerate the formation of the team or group so that they reach maximum productivity as early as possible.

History of the Fast Boot

The first Fast Boot took place in December 2018. The 13 members of Monitor

Group gathered for 3 days to work and bond in Berlin. You can learn more by reading the original planning issue.

The second Fast Boot took place in April 2019. The 5 members of Delivery team gathered in Utrecht to bond but also work on finalising auto-deployment process. Planning issue contains
the proposal for Fast boot, and
the Delivery Fast boot epic
contains issues and links to recordings created during the Fast Boot.

The third Fast Boot took place in Vancouver in September 2019. It included 18 people from Product, Engineering, UX and Data from the Acquisition, Conversion, Expansion and Retention teams. The planning issue contains the proposal for Fast Boot, and outcomes are available in the Growth Fast Boot Page.

Why should you have a Fast Boot?

Right now, the fast boot is intended for new teams or for teams with a majority of new members who need to build their culture of shipping work. If your team fits this description, you can propose holding a Fast Boot to reduce ramp up time and establish and strengthen relationships between team members.

How should you get approval?

First raise the idea with your manager to determine if there's a good case for a Fast Boot. Ultimately, the executive team will approve these on a case by case basis.

Development approval process

1. Create an issue outlining the proposed Fast Boot. Here is an example proposal from the Delivery Team Fast Boot.
1. Proposal is reviewed and approved by the Department head.
1. Proposal is reviewed and approved by the CTO.

How should you measure success?

When building the case for your Fast Boot, you should determine what metrics you will use to measure success. For example, after the Fast Boot you may expect your engineering team's throughput to increase or you may expect your support teams mean time to ticket resolution to decrease. You may also want to measure the sentiment of the team members after the Fast Boot with followup surveys.

What artifacts should you produce?

During the Fast Boot the priority is building a culture of shipping work. Ideally the group can work together to ship one large, high-impact item to production. Failing that it could be a backlog of smaller items. Secondarily you should record and livestream videos that give people insight into what your team is working on. Kickoff meetings, working sessions, and demos are all great candidates for being livestreamed.

Pros & Cons

Pros

Team members really enjoyed meeting each-other in person and got to spend a lot more time together than at the Summit/Contribute.
Team members now feel more comfortable asking for help, especially across teams (FE, BE, UX).
Team members understood each other's workflows better, allowing them to organise better and work more efficiently in the future.

Cons

It takes time to create a proposal and get the approval.
It takes a fair amount of planning: hotels, flights, meals, content, follow-up.
It can be expensive.
It is antithetical to the way we usually work at GitLab.
It can be mentally and physically exhausting spending a lot of time with team members outside of personal comforts.

Tips & Tricks

Hold the Fast Boot in a location where at least one of your team members resides. In addition to reducing the cost of flights and hotels, your team member can act as a guide to the city and help choose and book locations.
Set clear expectations about how your time will be spent. Will there be down time? Will the team eat dinner together every night?
If at all possible, select one day for non-work related activities on the tail end of the Fast Boot. Doing a fun activity will allow team members to get to know different aspects of ones personality.
Agree and plan in advance what events should be recorded and stick to the plan for the rest of the event.
Prepare any equipment for recording (or streaming) the events while getting approvals for Fast boot. Basic cameras might not be sufficient and additional equipment might need to be rented.
When booking accommodation, ensure that everyone gets enough personal space. Consider that everyone will spend most of the day working together, so personal downtime might be needed.

SECTION: engineering/gitlab-repositories.md

GitLab consists of many subprojects. A curated list of GitLab projects can be found at the GitLab Engineering projects page.

Creating a new project

When creating a new project, please follow these steps:

1. Read and familiarize yourself with our stance on Dogfooding. Be aware that as part of a product development organization that builds a tool for people like us, that our default is to add features and tooling to the GitLab project. This is still true when the effort to do so is 2-5x. Despite this, if you still feel you need to create a project outside of GitLab, you must follow this process to document the decision
1. Ensure the project is under a subgroup of:
   gitlab-org for anything related to the application.
   gitlab-com for anything strictly company related.

   To avoid complications with context and permissions inheritance, creating projects directly under these root namespaces (e.g. gitlab-org/NEWPROJECT) is discouraged. Only Maintainers can create projects there when necessary, but should also avoid doing so for the reason mentioned before.
   If you don't have the permissions to create a project there, you can create an Access Request issue and ping one of the Maintainers (gitlab-org, and gitlab-com) for approval.
1. Configure the project repository to use main as the name of the default branch.
1. Add the project to the list of GitLab projects in projects.yml.
1. Add a license to the repository. Contact legal as to which license to add. A sample license is here: gitlab-org/gitlab MIT License, but contact legal before using it.
1. Add a section titled "Developer Certificate of Origin and License" to CONTRIBUTING.md in the repository. It is easiest to simply copy-paste the gitlab-org/gitaly DCO + License section verbatim.
1. Add any further relevant details to the Contribution Guide. See Contribution Example.
1. Add a link to CONTRIBUTING.md from the project's README.md.
1. Add a CODEOWNERS file, to make it easy for contributors to figure out which teams are best suited to review their changes.
   Use teams rather than individuals as owners, to make it self updating over time and resilient to people taking time off
   You can scope ownership to subdirectories or individual files, but it should contain at the very least a top-level catch all for any new or non explicitly mentionned file.
1. When possible, projects should have the following Merge request settings enabled:
   Merge Trains.
   Delete source branch after merge.
   Merge only if pipeline succeeds.
   Merge only when all threads are resolved.
1. When possible, projects should have the following Pipeline settings enabled:
   Auto-cancel pending pipelines.
1. Projects should have the minimum Baseline Configurations setup for MR Approval Rules and Protected Branch Settings
1. Projects should have Users can request access setting disabled to discourage granting accidental external access.
1. If needed, make sure to set up a default CI/CD configuration.
1. If the project is part of work that is shipped to customers, add it to projectspartofproduct.csv by opening an MR to that file or following the process outlined by Engineering Productivity.
1. Help AppSec categorize your new project.
1. Enable the appropriate security scanners.
1. Onboard the project to Renovate (https://gitlab.com/gitlab-org/frontend/renovate-gitlab-bot), and set up a process to regularly triage findings and update dependencies
1. If the repository is public, set up a security mirror.

This is necessary to address security vulnerabilities without disclosing them before they are fixed.

When changing the settings in an existing repository, it's important to keep communication in mind. In addition to discussing the change in an issue and announcing it in relevant chat channels (e.g., development), consider announcing the change in the Engineering Week-in-Review document. This is particularly important for changes to the GitLab repository.

CI/CD configuration

Following is the default .gitlab-ci.yml config that all projects under the gitlab-org and gitlab-com groups should use:

yaml
include:
  - template: 'Workflows/MergeRequest-Pipelines.gitlab-ci.yml'

default:
  tags:
    - gitlab-org

Or if the project needs to support stable/security branches, use the following instead:

yaml
workflow:
  rules:
    For merge requests, create a pipeline.
    - if: '$CIMERGEREQUESTIID'
    For master branch, create a pipeline (this includes on schedules, pushes, merges, etc.).
    - if: '$CICOMMITBRANCH == $CIDEFAULTBRANCH'
    For tags, create a pipeline.
    - if: '$CICOMMITTAG'
    For stable, and security branches, create a pipeline.
    - if: '$CICOMMITBRANCH =~ /^[\d-]+-stable(-ee)?$/'
    - if: '$CICOMMITBRANCH =~ /^security\//'

default:
  tags:
    - gitlab-org

This:

1. Includes a workflow to create pipelines for MR, master, and tags only.
1. Defines the gitlab-org tag to be used by default which corresponds to cost-optimized runners, with no Docker support. Jobs that need Docker support would use the gitlab-org-docker tag.

If a job requires the usage of Docker, it needs to be defined only in the context of the

specific job with the gitlab-org-docker tag:

```yaml
sast:
  tags:
    - gitlab-org-docker
```

If a job requires the usage of Windows, SaaS runners on Windows should be used. For the exact configuration please check the SaaS runner on Windows documentation.

Publishing a Project

To publish a project to a package repository, please follow these directions.

Further Security Recommendations

1. Strongly consider creating a threat model for the project.
1. Consider requesting an AppSec review when the project is more established.
1. Reach out to the AppSec team (@gitlab-com/gl-security/appsec and sec-appsec) for any further questions.

SECTION: engineering/hiring.md

Overview

Hiring is a cornerstone of success for our engineering organization, contributing to our growth and our ability to drive results for our customers. As such, it's not just a responsibility but fundamental to every engineer's contribution to GitLab. It should be deeply ingrained in every engineer's role at GitLab, regardless of their seniority.

By actively participating in recruitment efforts, engineers help shape their team culture, elevate technical standards, and ensure a continuous influx of diverse perspectives and skillsets. Contributing to hiring efforts allows GitLab to grow responsibly and affects our collective success within Engineering.

Interviewing is a skill, and takes up a significant amount of time and effort to facilitate and improve upon. We should recognize and appreciate the efforts we take to contribute to scaling the engineering organization.

For more context, please review a Engineering All-Hands recording (internal) emphasizing the importance of contributing to hiring efforts within Engineering.

Hiring Practices

Please reference our internal hiring repository for internal best practices and guidelines.

R&D New Headcount GHPID Request, Backfill & Transfer Process

New Headcount and Backfill Processes

Budgeted new headcount review occurs on a regular basis across Engineering Leadership, Talent Acquisition, Finance Business Partners (FPB), and People Business Partners (PBP). The process to open a new headcount is Talent Acquisition and Finance-driven, with a focus on ensuring communication, consistency and collaboration between the Engineering and People teams.

The process for both new headcount and backfill processes can be found in the Talent Acquisition handbook section. Please note that we no longer require a Backfill Issue to be created and approved before we can start recruiting. Backfills are first reviewed on a weekly basis by the CTO and Engineering Leadership and a business decision is made to either open, close or repurpose the position. Your assigned recruiter will be in touch to Kickoff the role once a decision has been made. All backfills default to an Intermediate/Grade 6 level unless there is a business need to re-level, more information can be found in the Engineering Mobility Principles handbook page.

Headcount Transfer Process

1. Department Heads and PBPs: If two Department Heads agree to move a new headcount between their teams each leader should reach out to their respective PBPs to discuss org structure and/or org planning considerations. PBPs will share relevant information (org design/level/placement) with Finance and Talent Acquisition through the Triad process.
1. Talent Acquisition: TA will own the process of creating an Approved Needing Swap issue in the Headcount Project to confirm that both departments are in agreement with the move.
1. Finance Business Partner: The FBP will create a GHPID for the forecasted role in Adaptive, add the GHPID to the Issue and mark the step as complete, signaling the forecast has been updated. If more information is required, the FPB will reach out to the department leader or PBP before providing a GHPID.
1. People Business Partner: After a GHPID has been allocated, the PBP will ensure that both the "Cost Center" and "Supervisory Org" dimensions in Workday are updated and accurate.

SECTION: engineering/incident-management.md

Definition of an Incident

The definition of "incident" can vary widely among companies and industries. Here at GitLab, incidents are anomalous conditions that result in · or may lead to · service degradation, outages, or other disruptions. These events require human intervention to avert disruptions, communicate status, restore normal service, and identify future improvements.
Incidents are always given immediate attention.

Incident Management

Incident Management is the process of responding to, mitigating, and documenting an incident. At GitLab, we approach Incident Management as a feedback loop with the following steps, with different teams adjusting them as needed:

1. Preparation

The first step in an effective Incident Management program is preparation. This includes documentation of process and relevant training for everyone who could be involved in an incident. This step also includes ensuring the appropriate monitoring and alerting is in place, and the right people are part of the on-call rotation. The people involved in the preparation phase do not necessarily align with the roles for steps 2 - 7 defined below.

2. Identification

The various paths of identifying a problem include:

- Instrumentation/alerting/monitoring
- Customer reports
- Team member reports
- Security reports or other threat intelligence

Once a problem has been identified by one or more of the above paths, an incident is declared.

3. Investigation

Investigation includes looking for the cause of an outage/service disruption and an initial determination of the impact of the incident, which informs the severity level the incident is declared at. Severity levels may be changed as impact is further revealed in later stages.

4. Containment

Containing the impact and stabilizing the service as quickly as possible. Once containment is achieved and the impact of the disruption is alleviated, the incident is considered "mitigated".

5. Remediation

A more robust response to stabilizing the service. An incident is considered remediated, or "resolved", when all anomalous conditions are resolved.

6. Recovery

Improvements are made across testing and documentation, based on the specific containment and remediation actions taken for the incident. Corrective Actions, that have been identified to either prevent the incident from re-occurring or significantly improve future Time to Detection, Time to Mitigation or Time to Resolution of the Incident, may be started in this phase.

7. Learnings

This includes doing Root Cause Analysis, Incident Reviews/retrospectives, and identifying and implementing further Corrective Actions.
All of these should feed into updating documentation and training in step 1 and close the feedback loop.

Learning from incidents individually is important. Incidents should also be reviewed

holistically to identify trends and learnings in order to improve the organization's posture, processes, and product.

Incident roles

Role	Responsibilities
Incident Manager (IM)	Coordination of efforts across roles and teams.
Engineer On Call (EOC)	Responding to pages and conducting initial triage and investigation.
Communications Manager On Call (CMOC)	External communications through various channels.

On-call Schedule Management

For most on-call schedule management, GitLab uses PagerDuty to create schedules and set escalation policies.

We also employ a Development Escalation Process to get expertise from development teams as needed.

How we monitor and alert GitLab

Here is an overview on our monitoring. We use an in house tool to alert when a service is in breach of its SLI or SLO, which will also connect to PagerDuty.

SECTION: engineering/mentorship.md

Mentorship, Coaching and Engineering Programs

Senior Leaders in Engineering

The 7CTOs Program is run with 4 Senior leaders in Engineering. The program exists of peer mentoring sessions (forums) and effective network building.

How to find a mentor or become a mentor at GitLab

There is a program to find a mentor or to become a mentor at GitLab described on this handbook page.

Learning the customer experience

Engineers can gain significant insight from observing customers while they use the portions of the product that those engineers develop. Engineers interested in doing this should contact their team's assigned UX Researcher to get involved with upcoming customer sessions and access previously recorded sessions.

If a team does not have a UX Researcher assigned, they should reach out on the research Slack channel to express interest. A UX Researcher can jump in and set them up to observe a

participant session.

If there is a specific request to shadow a customer as they use the product, this should be coordinated through the team's assigned Product Manager.

SECTION: engineering/okrs.md

SECTION: engineering/on-call.md

{{% alert color="warning" %}}
If you're a GitLab team member and are looking to alert Reliability Engineering about an availability issue with GitLab.com, please find quick instructions to report an incident here: Reporting an Incident.
{{% /alert %}}

{{% alert color="warning" %}}
If you're a GitLab team member looking for who is currently the Engineer On Call (EOC), please see the Who is the Current EOC? section.
{{% /alert %}}

{{% alert color="warning" %}}
If you're a GitLab team member looking for help with a security problem, please see the Engaging the Security On-Call section.
{{% /alert %}}

Expectations for On-Call

- If you are on call, then you are expected to be available and ready to respond to PagerDuty pages as soon as possible, and within any response times set by our Service Level Agreements in the case of Customer Emergencies. If you have plans outside of your workspace during your on-call shift, this may require that you bring a laptop and reliable internet connection with you.
- We take on-call seriously. There are escalation policies in place so that if a first responder does not respond in time, another team member is alerted. Such policies are not expected to be triggered under normal operations, and are intended to cover extreme and unforeseeable circumstances.
- Because GitLab is an asynchronous workflow company, @mentions of On-Call individuals in Slack will be treated like normal messages, and no SLA for response will be associated with them.
- Provide support to the release managers in the release process.
- As noted in the main handbook, after being on-call, make sure that you take time off. Being available for issues and outages can be taxing, even if you had no pages. Resting after your on-call shift is critical for preventing burnout. Be sure to inform your team of the time you plan to take for time off.
  - Team members in Australia should review the Australia time in lieu policy.
- During on-call duties, it is the team member's responsibility to act in compliance with local

rules and regulations. If ever in doubt, please reach out to your manager and/or aligned People Business Partner.

Customer Emergency On-Call Rotation

- We do 7 days of 8-hour shifts in a follow-the-sun style, based on your location.
- After 10 minutes, if the alert has not been acknowledged, support management is alerted. After a further 5 minutes, everyone on the customer on-call rotation is alerted.
- All tickets that are raised as emergencies will receive the emergency SLA. The on-call engineer's first action will be to determine if the situation qualifies as an emergency and work with the customer to find the best path forward.
- After 30 minutes, if the customer has not responded to our initial contact with them, let them know that the emergency ticket will be closed and that you are opening a normal priority ticket on their behalf. Also let them know that they are welcome to open a new emergency ticket if necessary.
- You can view the schedule and the escalation policy on PagerDuty. You can also opt to subscribe to your on-call schedule, which is updated daily.
- After each shift, if there was an alert / incident, the on call person will send a hand off email to the next on call explaining what happened and what's ongoing, pointing at the right issues with the progress.
- If you need to reach the current on-call engineer and they're not accessible on Slack (e.g. it's a weekend, or the end of a shift), you can manually trigger a PagerDuty incident to get their attention, selecting Customer Support as the Impacted Service and assigning it to the relevant Support Engineer.
- See the GitLab Support On-Call Guide for a more comprehensive guide to handling customer emergencies.

GitLab.com Reliability On-Call Rotation

Infrastructure Engineer On-Call

The Infrastructure department's SREs provide 24x7 on-call coverage for the production environment. For details, please see incident-management.

In addition to incident management responsibilities, the EOC also is responsible for time sensitive interrupt work required to support the production environment that is not owned by another team. This includes:

1. Fulfilling Security Incident Response Team (SIRT) requests
1. Fulfilling Legal Preservation requests
1. Reviewing and handling certain change requests (CRs). This includes:
    1. Reviewing CRs to ensure they do not conflict with any ongoing incidents or investigations
    1. Executing the CR directly if the author does no thave the required permissions to make the change themselves (such as admin-level changes)
    1. Support during C1 CRs, such as database upgrades, that may occur on weekends
1. Handling urgent teleport access requests
1. Approving an exception for running ChatOps commands when they fail their safety checks
1. Investigating and fixing buggy/flapping alerts
1. Removing alerts that are no longer relevant

1. Collecting production information when requested
1. Responding to @sre-oncall Slack mentions
1. Assisting Release Managers with deployment problems
1. Being the DRI for incident reviews

Engineering Incident Manager

- Incident manager rotation is staffed by certain team members in the Development and Infrastructure departments.
- More information regarding the Incident Manager role, including shift schedules, responsibilities can be found in the Incident Manager on-boarding page.

Development Team On-Call Rotation

- This on-call process is designed for GitLab.com operational issues that are escalated by the Infrastructure team.
- Development team currently does NOT use PagerDuty for scheduling and paging. On-call schedule is maintained in this schedule sheet.
- Issues are escalated in the Slack channel dev-escalation by the Infrastructure team.
- First response SLO is 15 minutes. If no response within the first 5 minutes, the infrastructure team will call the engineer's phone number on the schedule sheet.
- Development engineers do 4-hour shifts.
- Engineering managers do monthly shifts as scheduling coordinators.
- Check out process description and on-call workflow when escalating GitLab.com operational issue(s).
- Check out more detail for general information of the escalation process.

Gitaly Engineer On-Call

For more details, see the team page

DBO On-Call

For more details, see the DBO escalation process

Security Team On-Call Rotation

Security Operations (SecOps)

- SecOps on-call rotation is 7 days of 24-hour shifts.
- After 15 minutes, if the alert has not been acknowledged, the Security Manager on-call is alerted.
- You can view the Security Operations schedule on PagerDuty.
- When on-call, prioritize work that will make the on-call better (that includes building projects, systems, adding metrics, removing noisy alerts). Much like the Production team, we strive to have nothing to do when being on-call, and to have meaningful alerts and pages. The only way of achieving this is by investing time in trying to automate ourselves out of a job.
- The main expectation when on-call is triaging the urgency of a page - if the security of GitLab is at risk, do your best to understand the issue and coordinate an adequate response. If you don't know what to do, engage the Security manager on-call to help you out.

- More information is available in the Security Operations On-Call Guide and the Security Incident Response Guide.

Security Managers

- Security Manager on-call rotation is 7 days of 12-hour shifts.
- Alerts are sent to the Security Manager on-call if the SecOps on-call page isn't answered within 15 minutes.
- You can view the Security Manager schedule on PagerDuty.
- The Security Manager on-call is responsible to engage alternative/backup SecOps Engineers in the event the primary is unavailable.
- In the event of a high-impact security incident to GitLab, the Security Manager on-call will be engaged to assist with cross-team/department coordination.

Developer Experience Stage On-Call Rotation

- Developer Experience's on-call do not include work outside GitLab's normal business hours. The process is defined on our pipeline on-call rotation page.
- The rotation is on a weekly basis across 3 timezones (APAC, EMEA, AMER) and triage activities happen during each team member's working hours.
- This on-call rotation is to ensure accurate and stable test pipeline results that directly affects our continuous release process.
- The list of pipelines which are monitored are defined on our pipeline page.
- The schedule and roster is defined on our schedule page.

PagerDuty

We use PagerDuty to set the on-call schedules, and to route notifications to the appropriate individual(s).

Swapping On-Call Duty

Team members covering a shift for someone else are responsible for adding the override in PagerDuty. This can be arranged in the eoc-general Slack channel. They can delegate this task back to the requestor, but only after explicitly confirming they will cover the requested shift(s). To set an override, click the "Schedule an Override" button from the side navigation on the Schedule page or after selecting the relevant block of time on the calendar or timeline view. This action defaults the person in the override to you — PagerDuty assumes that you're the person volunteering an override. If you're processing this for another team member, you'll need to select their name from the drop-down list. Also see this article for reference.

Adding and removing people from the roster

Whe